



UNIVERSITÉ NUMÉRIQUE CHEIKH HAMIDOU KANE
Licence Cybersécurité — Groupe 14

RAPPORT DE TRAVAUX PRATIQUES

LAB 1 — Pipeline CI/CD de sécurité

Jenkins × Bash × OWASP Juice Shop × DevSecOps

Module : Sécurité des données

Année académique : 2024-2025

Date : 10 août 2026

Groupe : 14

Dépôt GitHub : https://github.com/devmail0561-web/labs_sec_data

AUTEURS

MT

Michel TENDENG

michel.tendeng@unchk.edu.sn

MD

Mouhamed El Hadj Malick DIOP

mouhamedmalickidrisa.diop@unchk.edu.sn

⚠ **Cadre pédagogique strictement contrôlé.** Environnement local et isolé. Toute

1 Contexte et objectifs

Ce travail pratique s'inscrit dans le module **Sécurité des données** de la Licence Cybersécurité (UNCHK). Il se décompose en **deux parties** :

Partie	Objectif	Outils
Partie 1	Tests de sécurité automatisés sur Juice Shop via Jenkins	Jenkins, Bash, curl
Partie 2	CI/CD : récupération depuis Git + webhook + notifications	GitHub, pollSCM, emailtext

Environnement

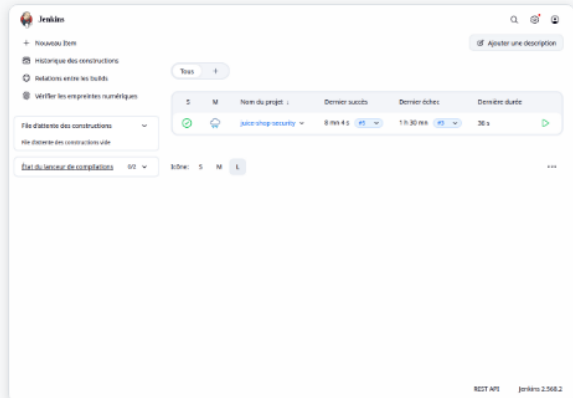
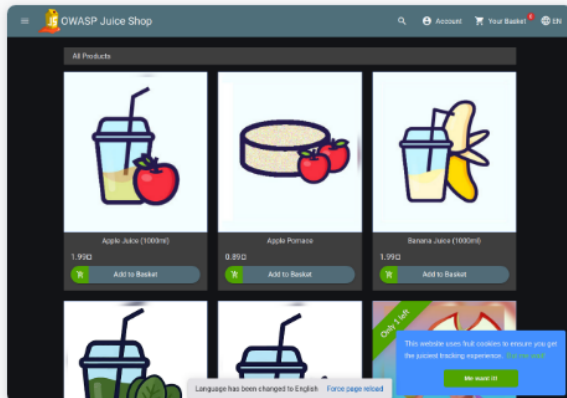
Composant	Valeur
Application cible	OWASP Juice Shop (Node.js / Express)
URL interne Docker	<code>http://juiceshop:3000</code>
Jenkins	<code>http://localhost:8080</code>
Réseau Docker	<code>172.20.0.0/24</code> — isolé
Dépôt Git	<code>https://github.com/devmail0561-web/labs_sec_data</code>
Branche	<code>main</code>

2 Architecture du laboratoire

```
Machine Linux (172.20.0.0/24)
├─ Jenkins :8080 (172.20.0.20)
│   └─ Source : GitHub (Pipeline from SCM)
│       │   URL   : https://github.com/devmail0561-web/labs_sec_data.git
│       │   Branch : main | Script : lab1-jenkins/Jenkinsfile
│   └─ Triggers
│       └─ pollSCM('H/5 * * * *') ← polling toutes les 5 min
```

2 Architecture du laboratoire

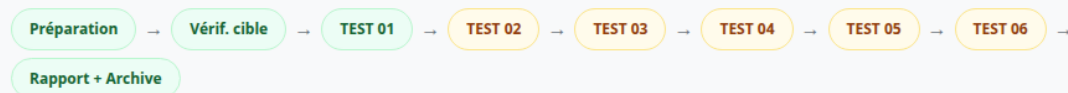
```
Machine Linux (172.20.0.0/24)
├─ Jenkins :8080 (172.20.0.20)
│   └─ Source : GitHub (Pipeline from SCM)
│       URL : https://github.com/devmail0561-web/labs_sec_data.git
│       Branch : main | Script : lab1-jenkins/Jenkinsfile
│
│   └─ Triggers
│       └─ pollSCM('H/5 * * * *') ← polling toutes les 5 min
│           └─ githubPush() ← webhook GitHub (push event)
│
│   └─ TEST 01 – Disponibilité HTTP
│   └─ TEST 02 – En-têtes de sécurité (A05:2021)
│   └─ TEST 03 – Méthodes HTTP (A05:2021)
│   └─ TEST 04 – Authentification + SQLi (A03, A07:2021)
│   └─ TEST 05 – Exposition APIs (A01, A02:2021)
│   └─ TEST 06 – Exploitation (SQLi/IDOR/FTP/XSS)
│   └─ Rapport consolidé + archiveArtifacts
│   └─ emailxnt → smtp.gmail.com:587
└─ Juice Shop :3000 (172.20.0.10)
    Scripts montés en /lab/scripts/ (lecture seule)
```



3 Partie 1 — Tests de sécurité automatisés

Structure du pipeline

Le `Jenkinsfile` définit **8 étapes**. Les étapes TEST 02 à TEST 06 utilisent `catchError(buildResult:'UNSTABLE')` : une vulnérabilité détectée marque l'étape en **FAILURE** mais laisse le pipeline continuer en **UNSTABLE**.



Build #5 — 10/08/2026 21:16 UTC — Statut final : **SUCCESS** (démarré par un changement SCM) — Durée : ~8 min

Test	Script	Résultat script	Statut Jenkins	OWASP
TEST 01	test_http.sh	FAILURE	SUCCESS (continu)	Baseline
TEST 02	test_headers.sh	FAILURE	UNSTABLE	A05:2021
TEST 03	test_methods.sh	FAILURE	UNSTABLE	A05:2021
TEST 04	test_auth.sh	FAILURE	UNSTABLE	A03, A07:2021
TEST 05	test_api_security.sh	FAILURE	UNSTABLE	A01, A02:2021
TEST 06	test_exploitation.sh	FAILURE	UNSTABLE	A01, A03:2021

TEST 01 Disponibilité HTTP — Baseline

Baseline

Endpoint	HTTP obtenu	Attendu	Statut
/	200	200	✓ PASS
/api/Challenges	200	200	✓ PASS
/api/Users	401	200	✗ FAIL
/rest/user/whoami	200	200	✓ PASS

4 Partie 2 — Automatisation CI/CD avec webhook

4.1 Pipeline from SCM

Le job a été reconfiguré de *script inline* vers **Pipeline from SCM**. Jenkins récupère le `Jenkinsfile` directement depuis GitHub à chaque build.

Type de définition	Pipeline from SCM (CpsScmFlowDefinition)
Repository URL	https://github.com/devmail0561-web/labs_sec_data.git
Credentials	Aucun (dépôt public)
Branch	*/main
Script Path	lab1-jenkins/Jenkinsfile

 Jenkins / Administrer Jenkins



Administrer Jenkins



Une construction sur le nœud principal peut engendrer des problèmes de sécurité. Vous devriez configurer des builds distribués.
[Consulter la documentation.](#)

[Set up agent](#)[Set up cloud](#)



Jenkins can enforce Content Security Policy (CSP). CSP tells web browsers what they are allowed to do while rendering a web page. This limits or even eliminates the impact of vulnerabilities like cross-site scripting (XSS). CSP is disabled by default for backward compatibility, but it is recommended to enable it, if possible.

[Learn more](#)

Configuration du système

**System**
Configurer les paramètres généraux et les chemins de fichiers.

**Tools**
Configurer les outils, leur localisation et les installeurs automatiques.

**Plugins**
Ajouter, supprimer, activer ou désactiver des plugins qui peuvent étendre les fonctionnalités de Jenkins.

**Nodes**
Ajouter, supprimer, contrôler et monitorer les divers nœuds que Jenkins utilise pour exécuter les jobs.

**Clouds**
Ajouter, supprimer et configurer les instances de cloud afin de provisionner les agents à la demande.

**Apparence générale**
Configurer l'apparence générale de Jenkins

Sécurité

5 Synthèse des résultats

ID	Vulnérabilité	OWASP	CVSS	Barre	Criticité	Exploitée
EXP-01	SQL Injection — Auth	A03:2021	9.8		● Critique	✓ Oui
EXP-02	IDOR — Paniers	A01:2021	8.1		● Critique	✓ Oui
TEST 05	Config admin sans auth	A01:2021	7.5		● Élevé	✓ Oui
EXP-03	Path Traversal + Null Byte	A01:2021	7.5		● Élevé	✓ Oui
EXP-04	XSS Stocké	A03:2021	7.2		● Élevé	⚠ Partiel
TEST 03	TRACE actif (XST)	A05:2021	5.8		● Moyen	—
TEST 02	En-têtes manquants	A05:2021	5.3		● Moyen	—
TEST 04	Absence rate limiting	A07:2021	5.3		● Moyen	—

Bilan CI/CD

Objectif	Statut	Preuve
Tests TEST 01-06	✓	6 scripts, builds #1-#5 exécutés
Artefacts archivés	✓	9 fichiers de preuves par build
Pipeline from SCM (GitHub)	✓	Checkout 9667c36 en console build #5
pollSCM (H/5)	✓	Build #5 démarré par SCM change
Webhook GitHub (githubPush)	✓	Déclaré — activer dans GitHub Settings
Notification email HTML	✓	emailx déclenché, SMTP credentials à finaliser

6 Recommandations

6 Recommandations

R01 Corriger les injections SQL Critique

Utiliser des requêtes préparées. Ne jamais interpoler les entrées utilisateur dans une requête SQL.

R02 Corriger les IDOR Critique

Vérifier la propriété : `if (basket.UserId !== req.user.id) return res.status(403).end()`. Utiliser des UUIDs non séquentiels.

R03 Ajouter les en-têtes de sécurité Moyen

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin
```

R04 Désactiver TRACE et méthodes dangereuses Moyen

```
app.use((req, res, next) => {
  if (['TRACE', 'TRACK'].includes(req.method)) return res.status(405).end()
  next()
})
```

R05 Implémenter le rate limiting Moyen

```
app.use('/rest/user/login', rateLimit({ windowMs: 15 * 60 * 1000, max: 10 })))
```

7 Conclusion

Partie 1 : Le pipeline Jenkins exécute automatiquement 6 batteries de tests à chaque build. Quatre vulnérabilités ont été exploitées avec preuves : SQLi → JWT admin (CVSS 9.8), IDOR paniers (CVSS 8.1), Path Traversal + null byte (CVSS 7.5), et tentative XSS stocké. Les rapports et preuves sont archivés automatiquement dans Jenkins à chaque build.

Partie 2 : Jenkins a été reconfiguré en *Pipeline from SCM* — checkout GitHub confirmé en console (commit `9667c36`, build #5 déclenché par SCM change). Le `pollSCM H/5` déclenche le pipeline en moins de 5 minutes après tout commit sur `main`. Le webhook `githubPush()` est déclaré pour déclenchement immédiat dès que Jenkins sera exposé. Les notifications email HTML sont envoyées en fin de build.

Ce pipeline illustre le concept **DevSecOps** : tout commit sur le dépôt déclenche automatiquement les tests de sécurité et notifie l'équipe — la sécurité devient un contrôle continu intégré au cycle de développement.

8 Annexes

Arborescence du dépôt

```
https://github.com/devmail0561-web/labs_sec_data
├── lab1-jenkins/
│   ├── Jenkinsfile                └─ Pipeline from SCM
│   ├── docker-compose.yml
│   ├── reports/
│   │   ├── rapport_lab1_jenkins.html
│   │   └── rapport_lab1_jenkins.md
│   └── scripts/
│       ├── test_http.sh           test_headers.sh
│       ├── test_methods.sh        test_auth.sh
│       ├── test_api_security.sh    test_exploitation.sh
│       ├── generate_report.sh
│       └── capture_screenshots.sh
```

Références

- [OWASP Top 10 2021](#)
- [OWASP Testing Guide v4.2](#)
- [CVSS v3.1 Calculator](#)

Ce pipeline illustre le concept **DevSecOps** : tout commit sur le depot declenche automatiquement les tests de securite et notifie l'equipe — la sécurité devient un contrôle continu intégré au cycle de développement.

8 Annexes

Arborescence du dépôt

```
https://github.com/devmail0561-web/labs_sec_data
├── lab1-jenkins/
│   ├── Jenkinsfile                └─ Pipeline from SCM
│   ├── docker-compose.yml
│   ├── reports/
│   │   ├── rapport_lab1_jenkins.html
│   │   └── rapport_lab1_jenkins.md
│   └── scripts/
│       ├── test_http.sh           test_headers.sh
│       ├── test_methods.sh       test_auth.sh
│       ├── test_api_security.sh   test_exploitation.sh
│       ├── generate_report.sh
│       └── capture_screenshots.sh
```

Références

- [OWASP Top 10 2021](#)
- [OWASP Testing Guide v4.2](#)
- [CVSS v3.1 Calculator](#)
- [Jenkins Pipeline Documentation](#)
- [Jenkins email-ext Plugin](#)
- [GitHub Webhooks](#)